PRD: PracticeOps Agent Console

1. Product Summary

Product name: PracticeOps Agent Console
Working tagline: Human-in-the-loop patient messaging and conversational back-office automation for small medical practices.

PracticeOps Agent Console is a multi-agent workflow system for small-to-medium medical practices. It gives clinics a white-labeled patient messaging assistant that can triage patient messages, draft safe responses, and route sensitive cases to staff or clinicians for review. It also gives back-office users a conversational command center to manage daily operational tasks across patient messages, scheduling, billing, payments, insurance, and finance-related workflows.

The product is designed for small-practice staff who are often overloaded and not highly technical. Instead of forcing users to navigate multiple complex dashboards, the system lets them ask operational questions in natural language and turns those requests into structured actions, tasks, drafts, and review queues.

This is not an autonomous medical advice bot. It is a human-in-the-loop clinic operations assistant that uses conversation as the interface and multiple agents as the backend workflow engine.

---

2. Product Thesis

Small and mid-sized medical practices are overloaded by patient communication and back-office coordination. Patient messages, scheduling requests, insurance questions, billing issues, payment questions, and clinical escalations often arrive through fragmented channels and require staff to manually determine what matters, who should handle it, and what action should happen next.

The core hypothesis is:

> A conversational interface can make clinic operations easier for non-technical back-office users, while a multi-agent backend safely classifies requests, retrieves approved information, drafts responses, creates tasks, and routes sensitive cases to humans.

The product should prove that AI can be useful not by replacing clinic staff or clinicians, but by reducing ambiguity, prioritizing work, and making the next step obvious.

---

3. Why This Product Now

Small-practice operations are increasingly moving toward AI-assisted workflows, but many products focus on isolated automation: scheduling, intake, reminders, procurement, or insurance verification. The

remaining opportunity is the conversational layer that connects these workflows and makes them usable for non-technical staff.

PracticeOps Agent Console focuses on two connected problems:

1. **Patient-facing communication overload**
   Patients ask clinics questions about scheduling, symptoms, procedures, cost, insurance, medications, and documents. Staff need to respond quickly, but not every message can be handled the same way. Clinical-risk messages need escalation, while routine messages can be drafted or routed.

2. **Back-office workflow complexity**
   Small-practice staff often need to manage patient messages, payments, expenses, insurance issues, billing follow-ups, and task assignments. These users may not want another complex dashboard. They need a simple way to ask, “What should I handle first?” or “Draft a response and assign this to billing.”

---

4. Goals

4.1 Product Goals

1. Let clinics configure a white-labeled patient assistant using clinic-specific settings, tone, staff roles, and approved knowledge sources.
2. Classify incoming patient messages into operational categories such as scheduling, billing, insurance, medication, procedure prep, symptom escalation, document request, or general question.
3. Identify risk level and determine whether a message can be drafted for routine staff approval or must be escalated to a clinician.
4. Generate safe, clinic-appropriate draft responses using only approved knowledge sources when needed.
5. Create structured staff tasks from patient messages and internal commands.
6. Provide a conversational back-office command center where staff can ask natural-language operational questions.
7. Demonstrate a real multi-agent workflow using Claude, replacing the previous Make.com-style orchestration with a code-based orchestrator.

4.2 Demo Goals for Friday

The Friday prototype does not need to be production-ready. It should demonstrate product thinking and working agent orchestration.

The demo should show:

1. A clinic-branded setup page.
2. A patient chat simulator.
3. A real Claude-powered patient-message analysis chain.

4. A staff review inbox with risk, routing, draft response, and action buttons.
5. A back-office command center where staff can ask what to handle first.
6. A working multi-agent architecture with structured JSON outputs.
7. Human-in-the-loop approval before sensitive messages are sent.

---

5. Non-Goals

The first version should not attempt to build:

1. Full EHR integration.
2. Real patient SMS or email sending.
3. Real payer portal automation.
4. Real payment collection.
5. Autonomous medical advice.
6. Full RCM automation.
7. Full scheduling automation.
8. Production HIPAA deployment.
9. Clinical diagnosis or treatment recommendation.
10. A complete Nitra product replacement.

The prototype should be framed as a product-thinking artifact, not as a finished healthcare system.

---

6. Target Users

6.1 Primary User: Small-Practice Back-Office Staff

Examples:

- Front-desk staff
- Office manager
- Billing coordinator
- Practice manager
- Doctor’s spouse or family member helping with operations
- Patient coordinator
- Small-clinic operations assistant

Characteristics:

- Often overloaded.
- May not be highly technical.
- Works across scheduling, patient communication, billing, insurance, and basic finance.
- Needs practical help, not complicated analytics.
- Wants to know what to do next.

6.2 Secondary User: Clinician or Clinical Reviewer

Examples:

- Physician
- Nurse
- Medical assistant
- Clinical supervisor

Characteristics:

- Does not want to review every routine patient message.
- Needs urgent or clinically sensitive issues surfaced clearly.
- Needs AI summaries that are concise and auditable.
- Must retain control over clinical-risk responses.

6.3 Tertiary User: Patient

Patients interact with the clinic-branded assistant.

Characteristics:

- Wants fast answers.
- May ask vague or mixed questions.
- May not know whether their issue is administrative, billing-related, or clinical.
- Trusts the clinic more than a generic AI vendor.

---

7. Core Use Cases

Use Case 1: Patient Message Triage

A patient sends a message:

> “My eye has been hurting since the injection yesterday. Should I wait?”

The system should:

1. Classify the message as a post-procedure symptom.
2. Mark it as high-risk clinical content.
3. Avoid giving direct medical advice.
4. Draft a safe escalation response.
5. Create an urgent clinician review task.
6. Place the message in the staff review inbox.

Use Case 2: Routine Scheduling Request

A patient sends:

> "Can I move my appointment to next week?"

The system should:

1. Classify the message as scheduling.
2. Mark it as low risk.
3. Draft a front-desk response.
4. Create a scheduling follow-up task.
5. Allow staff to approve or edit.

Use Case 3: Billing or Cost Question

A patient sends:

> "How much will my visit cost?"

The system should:

1. Classify the message as billing/cost.
2. Mark it as medium risk because patient financial information may require verification.
3. Draft a response that avoids unsupported estimates unless approved data is available.
4. Route to billing staff.
5. Create a follow-up task.

Use Case 4: Procedure Prep Question

A patient sends:

> "Do I need to fast before my injection tomorrow?"

The system should:

1. Classify the message as procedure preparation.
2. Retrieve approved clinic prep instructions.
3. Draft an answer only from approved content.
4. Require staff review if confidence is low or instructions are missing.

Use Case 5: Back-Office Morning Worklist

Staff asks:

> "What should I handle first this morning?"

The system should review mock clinic data and return a prioritized list:

1. Urgent clinical-risk patient messages.
2. Patient messages awaiting staff approval.
3. Billing/cost questions before upcoming appointments.
4. Unconfirmed appointments.
5. Finance or expense items needing review.

Use Case 6: Back-Office Action Command

Staff asks:

> "Draft a reply to Patient C and assign the billing follow-up to Maria."

The system should:

1. Understand the requested action.
2. Identify the patient/message.
3. Generate a draft response.
4. Create a billing follow-up task assigned to Maria.
5. Mark the action as pending human approval.

---

8. Product Scope

8.1 MVP Scope

The MVP should include:

1. Clinic setup/configuration.
2. Patient chat simulator.
3. Patient message analysis workflow.
4. Staff review inbox.
5. Back-office command center.
6. Task creation simulation.
7. Human approval state.
8. Mock clinic data.
9. Actual Claude API calls for at least the patient-message workflow.
10. Structured agent outputs.

8.2 Post-MVP Scope

Later versions could add:

1. Real message-channel integrations.
2. EHR/PMS integration.
3. Scheduling API integration.

4. Billing system integration.
5. Payment-link generation.
6. Document upload and parsing.
7. Audit logs.
8. Clinic-specific knowledge ingestion.
9. Analytics dashboard.
10. Role-based access control.

---

9. Key Product Principles

9.1 Human-in-the-Loop by Default

The AI should not autonomously send clinical-risk messages. It should classify, draft, route, and recommend actions, but humans approve sensitive outputs.

9.2 Conversation as Workflow Interface

The chat interface is not just for casual interaction. It is the main command interface for non-technical clinic users to operate workflows.

9.3 Multi-Agent by Responsibility

Agents should not exist just to sound advanced. Each agent should map to a real responsibility:

- intent classification
- safety and escalation
- knowledge retrieval
- action planning
- drafting
- validation

9.4 Approved Knowledge Only for Patient-Facing Answers

For procedure prep, clinic policy, billing policy, and other informational questions, the assistant should use approved clinic content rather than open-ended medical knowledge.

9.5 No Autonomous Medical Advice

The system should avoid diagnosis, treatment recommendation, or reassurance for clinical symptoms. It should escalate and draft safe responses.

9.6 Action-Oriented Outputs

Every analysis should produce a practical next step: approve, edit, assign, escalate, call, message, review, or resolve.

---

10. Functional Requirements

10.1 Clinic Setup

The clinic setup screen should allow configuration of:

- clinic name
- specialty
- assistant name
- brand/tone
- staff roles
- escalation rules
- approved knowledge sources
- review rules

Example configuration:

```
{
  "clinic_name": "Ann Arbor Retina Clinic",
  "specialty": "Ophthalmology",
  "assistant_name": "ArborCare Assistant",
  "tone": "warm, concise, professional",
  "review_rules": {
    "clinical_symptom": "clinician_review_required",
    "billing_dispute": "billing_manager_review_required",
    "scheduling": "front_desk_review"
  }
}
```

10.2 Patient Chat Simulator

The patient chat simulator should allow the user to enter a patient message and trigger the agent workflow.

Required features:

- text input for patient message
- sample scenario buttons
- “Run Agent Workflow” button
- display final patient-facing draft
- display whether human review is required

10.3 Staff Review Inbox

The review inbox should show analyzed patient messages as cards.

Each card should include:

- patient alias
- original message
- category
- risk level
- route-to role
- AI draft response
- suggested next action
- review status
- buttons: approve, edit, assign, escalate, mark resolved

10.4 Back-Office Command Center

The command center should allow staff to ask natural-language operational questions.

Example commands:

- “What should I handle first this morning?”
- “Show me all unresolved patient messages.”
- “Draft a response to Patient B.”
- “Create a billing task for Maria.”
- “Which items need clinician review?”
- “Which finance items are still unresolved?”

Output should include:

- prioritized worklist
- recommended actions
- generated drafts
- created tasks
- pending approvals

10.5 Task System

The MVP task system can be simulated in local state or mock JSON.

Task fields:

- task ID
- title
- source message or command
- assigned role/person
- priority
- status
- due date, optional
- created by AI or human
- approval required

10.6 Human Approval

The system should support the following states:

- AI drafted
- needs review
- approved
- edited
- assigned
- escalated
- resolved

For the prototype, these can be UI state changes without persistence.

---

11. Agent Architecture

11.1 Patient Message Workflow

```
Patient message
    ↓
Orchestrator
    ↓
Intent Agent
    ↓
Safety & Escalation Agent
    ↓
Knowledge Agent
    ↓
Action Planner Agent
    ↓
Drafting Agent
    ↓
QA / Validation Agent
    ↓
Human review inbox
```

11.2 Agent 1: Intent Agent

Purpose:

Classify the user message.

Input:

- patient message
- clinic specialty

Output:

```
{
  "primary_intent": "post_procedure_symptom",
  "secondary_intents": [],
  "domain": "clinical",
  "confidence": 0.92,
  "summary": "Patient reports eye pain after injection and asks whether to
wait."
}
```

Possible intents:

- scheduling
- billing_cost
- insurance
- medication_refill
- procedure_prep
- post_procedure_symptom
- document_upload
- general_question
- complaint
- unknown

11.3 Agent 2: Safety & Escalation Agent

Purpose:

Decide risk level and review requirement.

Output:

```
{
  "risk_level": "high",
  "needs_human_review": true,
  "route_to": "clinician",
  "allowed_response_type": "safety_escalation_only",
  "safety_reason": "Patient reports post-procedure eye pain; assistant should
```

```
not diagnose, reassure, or advise waiting."
}
```

Risk levels:

- low
- medium
- high
- emergency

11.4 Agent 3: Knowledge Agent

Purpose:

Retrieve relevant approved clinic knowledge.

For MVP, knowledge can be passed as static mock policy content.

Output:

```
{
  "matched_sources": [
    {
      "title": "Post-injection symptom escalation policy",
      "relevance": "high",
      "allowed_content": "Escalate reports of eye pain, sudden vision changes,
severe redness, or worsening symptoms to clinical staff. Do not diagnose or
reassure."
    }
  ],
  "knowledge_confidence": "high"
}
```

11.5 Agent 4: Action Planner Agent

Purpose:

Create operational next actions.

Output:

```
{
  "recommended_actions": [
    {
      "type": "create_task",
```

```
      "title": "Clinician review for post-injection eye pain",
      "assignee_role": "clinician",
      "priority": "urgent",
      "requires_approval": false
    },
    {
      "type": "draft_patient_reply",
      "title": "Draft escalation response",
      "requires_approval": true
    }
  ]
}
```

11.6 Agent 5: Drafting Agent

Purpose:

Generate the patient-facing draft in clinic tone.

Output:

```
{
  "draft_reply":
"I'm sorry you're experiencing this. I'm going to flag this for the clinical
team now. If you have severe pain, sudden vision changes, or rapidly worsening
symptoms, please contact the clinic immediately or seek urgent care.",
  "tone": "warm, concise, professional",
  "requires_approval": true
}
```

11.7 Agent 6: QA / Validation Agent

Purpose:

Validate the final output.

Checks:

- Did the draft provide medical advice?
- Did the draft stay within approved knowledge?
- Was the escalation route appropriate?
- Is human review required?
- Is the tone appropriate?

Output:

```
{
  "approved_for_review_queue": true,
  "issues": [],
  "final_status": "needs_clinician_review"
}
```

---

12. Back-Office Command Workflow

12.1 Workflow

```
Staff command
    ↓
Command Intent Agent
    ↓
Mock Data Retrieval Agent
    ↓
Priority Agent
    ↓
Action Planner Agent
    ↓
Draft / Task Generator
    ↓
Command Center Output
```

12.2 Example Input

> “What should I handle first this morning?”

12.3 Example Output

```
{
  "worklist": [
    {
      "priority": 1,
      "item": "Patient A reported eye pain after injection",
      "reason": "High-risk clinical message requiring clinician review",
      "recommended_action": "Assign clinician review immediately"
    },
    {
      "priority": 2,
      "item": "Patient C asked about cost before tomorrow’s visit",
      "reason": "Unresolved cost question may cause cancellation",
```

```
      "recommended_action": "Assign billing follow-up"
    },
    {
      "priority": 3,
      "item": "Medical Supply Co. card transaction needs category review",
      "reason": "Month-end reconciliation item remains unresolved",
      "recommended_action": "Ask office manager to categorize transaction"
    }
  ]
}
```

---

13. Demo Dataset

13.1 Mock Clinic

```
{
  "clinic_name": "Ann Arbor Retina Clinic",
  "specialty": "Ophthalmology",
  "assistant_name": "ArborCare Assistant",
  "staff": [
    { "name": "Maria", "role": "billing" },
    { "name": "Dr. Lee", "role": "clinician" },
    { "name": "Sam", "role": "front_desk" }
  ]
}
```

13.2 Mock Patient Messages

1. “My eye has been hurting since the injection yesterday. Should I wait?”
2. “Can I move my appointment to next week?”
3. “How much will my visit cost?”
4. “Do I need to fast before my injection tomorrow?”
5. “Did you receive my new insurance card?”
6. “Can I get a refill for my eye drops?”

13.3 Mock Back-Office Items

1. Patient A: post-procedure symptom, urgent clinician review.
2. Patient B: rescheduling request, front-desk follow-up.
3. Patient C: cost question before appointment, billing follow-up.
4. Patient D: procedure prep question, approved FAQ available.
5. Finance item: card transaction needs category review.
6. Finance item: bill payment pending approval.

---

14. User Experience Requirements

14.1 UI Layout

Recommended layout:

- Left sidebar navigation
- Clinic Setup
- Patient Chat Simulator
- Staff Review Inbox
- Back-Office Command Center
- Tasks
- Main content area
- Right-side detail/action drawer

14.2 Design Tone

The UI should feel:

- professional
- clean
- healthcare-admin oriented
- trustworthy
- operational
- not overly futuristic
- easy for non-technical users

14.3 Important UI Patterns

Use:

- status badges
- risk labels
- priority cards
- clear next-action buttons
- human approval states
- source/knowledge references when used
- short reasoning summaries, not hidden chain-of-thought

Avoid:

- overly technical logs in the main UI
- black-box AI output
- autonomous send buttons for clinical-risk messages
- medical diagnosis language

---

15. Technical Requirements

15.1 Recommended Stack

- Frontend: Next.js
- Styling: Tailwind CSS
- Components: shadcn/ui or custom components
- Backend: Next.js API routes
- LLM: Claude API
- Data: local mock JSON for prototype; Supabase later
- Hosting: Vercel later

15.2 Suggested File Structure

```
/app
  /api
    /analyze-message
      route.ts
    /backoffice-command
      route.ts
  /page.tsx
/components
  ClinicSetup.tsx
  PatientChatSimulator.tsx
  StaffReviewInbox.tsx
  BackOfficeCommandCenter.tsx
  TaskPanel.tsx
/lib
  /agents
    intentAgent.ts
    safetyAgent.ts
    knowledgeAgent.ts
    actionPlannerAgent.ts
    draftingAgent.ts
    validationAgent.ts
    commandIntentAgent.ts
    priorityAgent.ts
  orchestrator.ts
  backofficeOrchestrator.ts
  mockData.ts
  claude.ts
/types
  index.ts
```

15.3 Claude Output Format

All agent calls should return structured JSON.

The orchestrator should validate outputs before passing them to the next agent.

15.4 Error Handling

The prototype should handle:

- Claude API failure
- invalid JSON
- missing fields
- low confidence classification
- unavailable knowledge source

Fallback behavior:

- show “Needs human review”
- avoid generating patient-facing final answer
- ask staff to manually review

---

16. Safety and Compliance Requirements

16.1 Patient-Facing Safety Rules

The assistant must not:

- diagnose
- tell patients to wait when symptoms may be serious
- provide treatment recommendations
- change medication instructions
- provide unsupported cost estimates
- claim clinician approval without human review

The assistant may:

- acknowledge the patient’s concern
- route to staff or clinician
- provide clinic-approved general instructions
- recommend contacting the clinic or urgent care for red-flag symptoms
- draft messages for human approval

16.2 Human Review Rules

Human review required for:

- clinical symptoms
- medication questions
- post-procedure complications
- billing disputes
- insurance denial questions
- any low-confidence classification
- any response using incomplete knowledge

16.3 Auditability

Every AI output should include:

- original message
- classification
- risk level
- route-to role
- draft response
- suggested action
- whether human review is required
- short reason summary

---

17. Success Metrics

17.1 Prototype Success Metrics

The Friday prototype is successful if it can demonstrate:

1. Real multi-agent Claude workflow.
2. Clear separation of intent, safety, knowledge, action, and drafting.
3. Human-in-the-loop safety design.
4. Conversational command center for back-office users.
5. A believable small-practice workflow.
6. Strong alignment with Nitra-style small-practice operations.

17.2 Future Product Metrics

Potential future metrics:

- reduction in patient message handling time
- percentage of routine messages drafted automatically
- percentage of high-risk messages correctly escalated
- staff response time improvement

- number of tasks created from patient messages
- number of unresolved back-office items surfaced by command agent
- staff satisfaction
- reduction in missed follow-ups

---

18. Open Questions

1. Should the first target user be front-desk staff, billing staff, or office manager?
2. Should the first specialty be ophthalmology, dermatology, med spa, or general small practice?
3. Should the patient-facing assistant be central, or should the internal back-office command center be central?
4. How much should the prototype emphasize Nitra-adjacent fintech workflows like card, bill pay, and expense management?
5. Should the system show agent-by-agent outputs in the UI, or hide them behind a “View analysis” panel?
6. Should the prototype use real Claude calls for both workflows, or only for patient-message analysis?
7. What is the safest way to describe this product in the Friday conversation without sounding like it is trying to replace Nitra’s current roadmap?

---

19. Recommended Friday Positioning

Use this framing:

> I built a lightweight prototype to pressure-test my understanding of the opportunity Greg described: small-practice users are often non-technical, and conversation can become an interface for both patient-facing and back-office workflows. The prototype is not meant to be a complete product or a recommendation for Nitra’s roadmap. It is a way to demonstrate how I think about agent design, safety boundaries, human review, and workflow automation in healthcare.

Key points to emphasize:

1. This is not autonomous medical advice.
2. This is not simply a scheduling bot.
3. This is not trying to duplicate Nitra’s Patient Management product.
4. The focus is the human-in-the-loop conversation layer.
5. The internal command center demonstrates back-office automation for non-technical users.
6. The multi-agent system is designed around real responsibilities, not arbitrary agent complexity.

---

20. MVP Build Checklist

Must Build

- [ ] App shell with sidebar navigation.
- [ ] Clinic setup/config screen.
- [ ] Patient chat simulator.
- [ ] `/api/analyze-message` endpoint.
- [ ] Intent agent.
- [ ] Safety agent.
- [ ] Knowledge agent.
- [ ] Action planner agent.
- [ ] Drafting agent.
- [ ] Validation agent.
- [ ] Staff review inbox.
- [ ] Task creation UI.
- [ ] Back-office command center.
- [ ] Mock data.
- [ ] One or two polished demo scenarios.

Nice to Build

- [ ] Agent output trace panel.
- [ ] Risk badge UI.
- [ ] Human approval workflow.
- [ ] Editable draft response.
- [ ] Mock finance/expense items.
- [ ] Back-office command agent using real Claude call.

Do Not Build Yet

- [ ] Real patient messaging.
- [ ] Real scheduling API.
- [ ] Real payment API.
- [ ] Real EHR/PMS connection.
- [ ] Real PHI storage.
- [ ] Production auth.

---

21. One-Sentence Product Definition

PracticeOps Agent Console is a human-in-the-loop, multi-agent conversational workflow system that helps small medical practices triage patient messages, draft safe responses, assign staff tasks, and automate routine back-office operations through natural language.